Vertiefung Echtzeitsysteme

Laborübung 2 – RTEMS Thread- Analyse und Capture

Eingereicht von

Mazen Younis

Fachbereich VI – Technische Informatik – Embedded Systems (M. Eng.)
Berliner Hochschule für Technik

Abgabedatum: 26.05.2026

Betreuer
Prof. Dr. Heinz Junkes Berliner Hochschule für Technik

BHT Berliner Hochschule für Technik

Studiere Zukunft

Laborübung 2 – RTEMS Thread-Analyse

1 RTEMS Thread-Analyse und Capture

1.1 Statische Quellcode-Analyse des Programms `psx_example_3`

Im Quellcode werden zwei POSIX-Threads mit `pthread_create()` erzeugt. Zusätzlich existiert der Hauptthread, der unter RTEMS über `POSIX_Init()` gestartet wird.

Thread	Startfunktion	Beschreibung
`child1`	`print_hello()`	Höchste `SCHED_FIFO`-Priorität, wartet anschließend mittels `pthread_cond_timedwait()` auf ein Timeout.
`child2`	`print_hello_a()`	Um eins niedrigere Priorität, gibt eine kurze Meldung aus und beendet sich.
`main`	`POSIX_Init()/main()`	Initialisiert Ressourcen, erzeugt Threads und wartet mit `pthread_join()` auf deren Beendigung.

Tab. 1: Identifizierte Threads in `psx_example_3`

Nur `child1` und `child2` werden explizit mit `pthread_create()` erzeugt. Der Hauptthread stellt den initialen Ausführungskontext dar.

Thread-Attribute

Die Threads verwenden ein gemeinsames Attributobjekt `pthread_attr_t attr`. Die Scheduling-Policy wird auf `SCHED_FIFO` gesetzt, wodurch Threads strikt nach Priorität ausgeführt werden.

- `child1`: `sched_get_priority_max(SCHED_FIFO)`
- `child2`: `sched_get_priority_max(SCHED_FIFO)-1`
- Stackgröße: nicht explizit definiert → RTEMS-Standardwert

Synchronisationsmechanismen

Im Programm werden folgende Mechanismen verwendet:

- `pthread_mutex_t`: notwendiger Parameter für `pthread_cond_timedwait()`
- `pthread_cond_t`: blockiert `child1` bis zum Timeout
- `pthread_join()`: Hauptthread wartet auf beide Threads

Da kein `pthread_cond_signal()` verwendet wird, endet das Warten ausschließlich durch das Timeout.

```
1 pthread_mutex_t mutex;
2 pthread_cond_t cond;
3
```

```
4 pthread_cond_timedwait(&cond,&mutex,&timeout);
5
6 pthread_join(child1,NULL);
7 pthread_join(child2,NULL);
```

Zustandsmodell

Für die beiden Threads ergeben sich folgende Zustandsübergänge:

`child1:`

New → Ready → Running → Blocked → Ready → Running → Terminated

`child2:`

New → Ready → Running → Terminated

Die Zustandsübergänge sind zusätzlich in Abbildung 1 dargestellt. Besonders `child1` durchläuft den Zustand *Blocked*, da der Thread innerhalb von `pthread_cond_timedwait()` auf ein Timeout wartet.

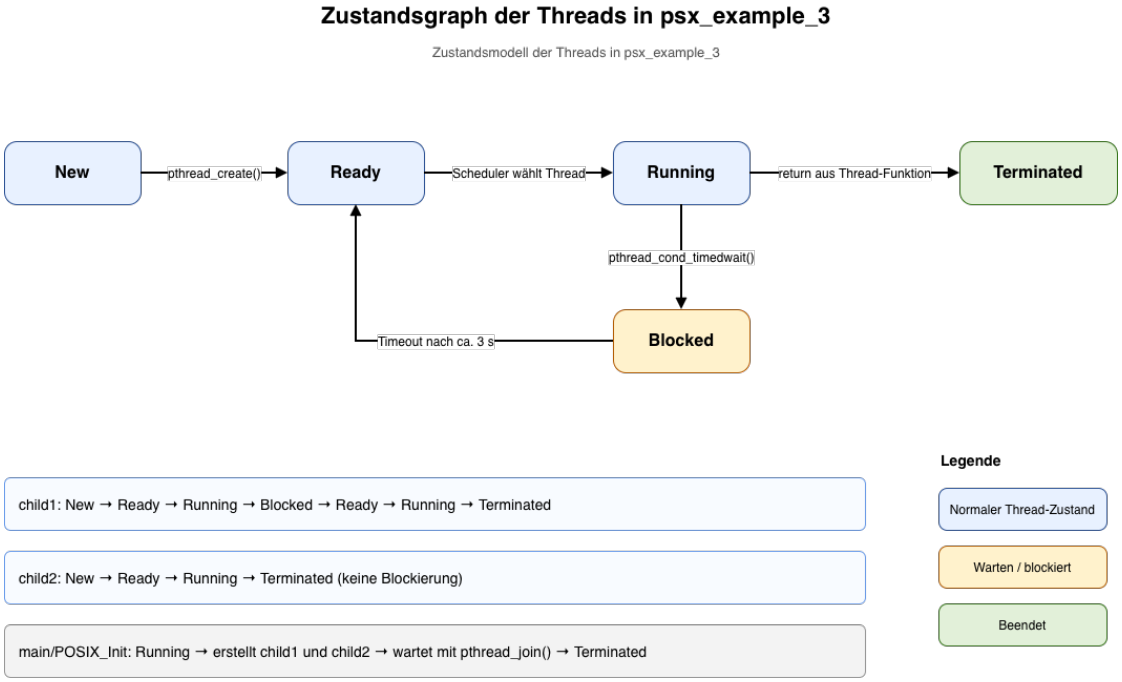


Abb. 1: Zustandsmodell der Threads in `psx_example_3`

Ausführungsreihenfolge

Durch `SCHED_FIFO` und die höhere Priorität läuft `child1` zuerst. Nach dem Aufruf von `pthread_cond_timedw` wird der Thread blockiert, sodass der Hauptthread `child2` erzeugen kann. Nach Ablauf des Timeouts wird `child1` erneut ausgeführt und beendet sich.

Die beobachtete Reihenfolge lautet:

- `child1`
- `child2`
- `child1` (nach Timeout)

Dies stimmt mit der späteren GDB-Analyse überein.

2 Dynamische Thread-Analyse mit GDB und QEMU

Zur Analyse des Laufzeitverhaltens wurde `psx_example_3` unter QEMU im Debug-Modus gestartet und mit GDB untersucht. Ziel war die Verifikation der statisch ermittelten Thread-Reihenfolge und Zustandswechsel.

2.1 Start von QEMU und Verbindung mit GDB

QEMU wurde mit aktiviertem GDB-Server gestartet:

```
EXE=build/arm-rtems7-xilinx_zynq_a9_qemu/
posix_api/psx_example_3/psx_example_3.exe

qemu-system-arm \
 -M xilinx-zynq-a9 \
 -m 256M \
 -nographic \
 -kernel $EXE \
 -s -S
```

Die Option `-s` startet den GDB-Server auf Port 1234, während `-S` die CPU zunächst anhält.
Die Verbindung erfolgte anschließend über:

```
arm-rtems7-gdb \$EXE
(gdb) target remote :1234
```

2.2 Verwendung von scheduler-locking

Für die schrittweise Untersuchung einzelner Threads wurde:

```
(gdb) set scheduler-locking on
```

verwendet.

Dadurch führt GDB nur den aktuell ausgewählten Thread weiter aus. Für die Beobachtung des normalen Scheduling-Verhaltens wurde anschließend:

```
(gdb) set scheduler-locking off
```

verwendet.

2.3 Analyse der Thread-Erzeugung und Synchronisation

Zur Laufzeitanalyse wurden Breakpoints auf zentrale Funktionen gesetzt:

```
(gdb) break POSIX_Init
(gdb) break pthread_create
(gdb) break pthread_cond_timedwait
(gdb) break pthread_join
```

Der erste Breakpoint bestätigte den Einstiegspunkt:

```
Breakpoint 1, POSIX_Init(arg=0x0)
at test3.c:93
```

Anschließend zeigte die Analyse von `pthread_create()`, dass zunächst `child1` und später `child2` erzeugt werden.

```
Breakpoint pthread_create
start_routine=<print_hello>
```

Während der Ausführung von `child1` wurde folgender Breakpoint erreicht:

```
Breakpoint pthread_cond_timedwait(...)
#1 print_hello()
```

Zusätzlich wurde an relevanten Breakpoints ein Backtrace sowie die Thread-Übersicht verwendet:

```
(gdb) bt
(gdb) info threads
```

Der Backtrace zeigte den aktuellen Aufrufpfad, während `info threads` den aktuellen Thread-Kontext und den Ausführungszustand bestätigte.

Dies zeigt, dass sich `child1` innerhalb von `print_hello()` befindet und anschließend durch `pthread_cond_timedwait()` blockiert wird.

Da `SCHED_FIFO` verwendet wird, übernimmt anschließend der Hauptthread wieder die CPU und erzeugt `child2`.

Am Ende wurde zusätzlich `pthread_join()` analysiert:

```
Breakpoint pthread_join(...)
#1 main()
```

Dadurch wurde bestätigt, dass der Hauptthread auf die vollständige Terminierung beider Threads wartet.

2.4 Einfluss der QEMU-Ausführung

Die Ausführung erfolgte unter QEMU auf dem BSP `xilinx_zynq_a9_qemu`. Innerhalb dieser Laborumgebung wurde das Verhalten effektiv als Single-Core-System betrachtet. Dadurch werden Threads nicht parallel, sondern ausschließlich durch Kontextwechsel des RTEMS-Schedulers ausgeführt.

Dies erleichtert die Analyse, da Zustandswechsel eindeutig dem Scheduling-Verhalten zugeordnet werden können.

2.5 Zusammenfassung

Die GDB-Analyse bestätigte die Ergebnisse der statischen Untersuchung:

- `POSIX_Init` startet den Hauptablauf
- `child1` wird zuerst erzeugt
- `child1` besitzt die höchste Priorität
- `child1` blockiert durch `pthread_cond_timedwait()`
- währenddessen wird `child2` ausgeführt
- der Hauptthread wartet mittels `pthread_join()`

Damit stimmen die beobachteten Laufzeiteigenschaften mit der statischen Analyse und dem erwarteten Verhalten von `SCHED_FIFO` überein.

Das vollständige GDB-Protokoll befindet sich in Anhang A.

3 Sequenzdiagramm und Synchronisationsanalyse

Zur Ergänzung der GDB-Laufzeitanalyse wurde ein Sequenzdiagramm erstellt. Das Diagramm visualisiert die zeitliche Reihenfolge der Thread-Erzeugung, Synchronisation und Zustandswechsel im Programm `psx_example_3`.

3.1 Analyse der Thread-Interaktion

Die Reihenfolge der Ereignisse wurde anhand der GDB-Analyse rekonstruiert. Dabei wurden insbesondere die Funktionen `pthread_create()`, `pthread_cond_timedwait()` und `pthread_join()` betrachtet.

Zunächst erzeugt `main()` den Thread `child1`:

```
1 Breakpoint pthread_create
2 start_routine=<print_hello>
```

Da `child1` die höchste `SCHED_FIFO`-Priorität besitzt, wird dieser unmittelbar ausgeführt.

Während seiner Ausführung erreicht `child1`:

```
1 Breakpoint pthread_cond_timedwait(...)
2 #1 print_hello()
```

Dadurch wird `child1` blockiert und in den Wartezustand der Condition Variable versetzt. Während der Wartephase gibt der Thread die CPU frei.

Da im Programm kein

```
1 pthread_cond_signal()
2 pthread_cond_broadcast()
```

verwendet wird, endet das Warten ausschließlich durch das definierte Timeout.

Während `child1` blockiert ist, setzt der Scheduler die Ausführung des Hauptthreads fort. Dieser erzeugt anschließend `child2`:

```
1 Breakpoint pthread_create
2 start_routine=<print_hello_a>
```

Zum Abschluss synchronisiert sich der Hauptthread über:

```
1 pthread_join(child1,NULL);
2 pthread_join(child2,NULL);
```

Dadurch wird sichergestellt, dass das Programm erst endet, nachdem beide Threads vollständig terminiert wurden.

3.2 Sequenzdiagramm der Thread-Interaktion

Die beobachtete Interaktion der Threads ist in Abbildung 2 dargestellt.

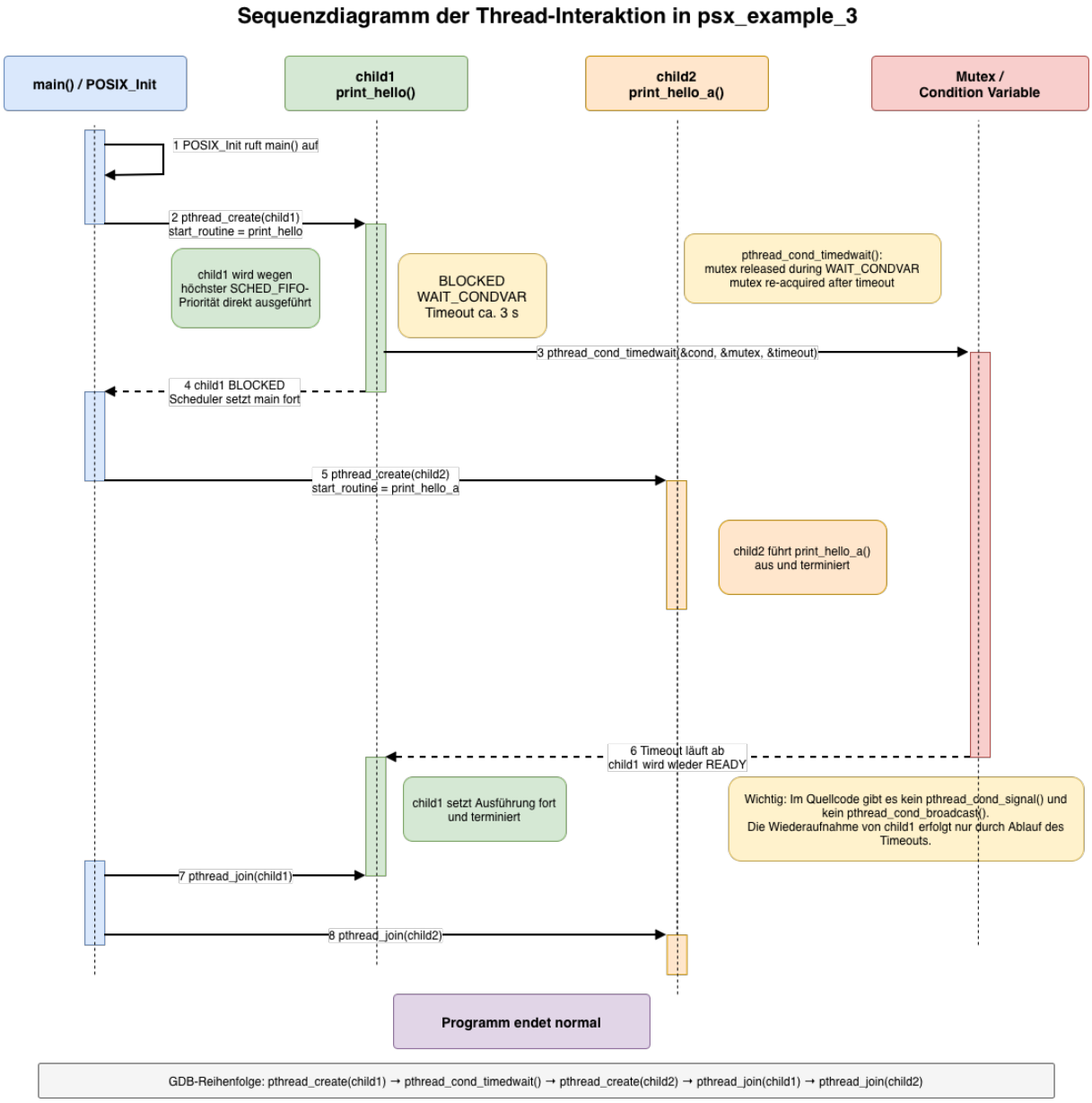


Abb. 2: Sequenzdiagramm der Thread-Interaktion in `psx_example_3`

Das Sequenzdiagramm zeigt die zeitliche Abfolge der Interaktionen zwischen Hauptthread, `child1`, `child2` sowie den verwendeten Synchronisationsmechanismen.

Zunächst erzeugt `main()` den Thread `child1`. Aufgrund seiner höheren Priorität wird dieser sofort ausgeführt und erreicht `pthread_cond_timedwait()`.

Während der Wartephase befindet sich `child1` im Zustand *Blocked*. Da kein Signal gesendet wird, erfolgt die Wiederaufnahme ausschließlich durch das Timeout.

Währenddessen erzeugt der Hauptthread `child2`, welcher seine Funktion vollständig ausführt und ohne Blockierung terminiert.

Nach Ablauf des Timeouts wird `child1` erneut ausgeführt und beendet sich regulär. Anschließend führt der Hauptthread `pthread_join(child1)` und `pthread_join(child2)` aus, sodass beide Threads vor Programmende berücksichtigt werden.

3.3 Zusammenfassung

Die Sequenzanalyse bestätigt das zuvor beobachtete Laufzeitverhalten:

- `main()` erzeugt zuerst `child1`
- `child1` wird unmittelbar ausgeführt
- `child1` blockiert durch `pthread_cond_timedwait()`
- währenddessen wird `child2` erzeugt
- `child2` terminiert ohne Wartezustand
- nach dem Timeout läuft `child1` weiter
- der Hauptthread wartet mittels `pthread_join()`

Damit stimmt die Darstellung im Sequenzdiagramm mit dem beobachteten Verhalten unter `SCHED_FIFO` überein.

4 Task-Analyse des `capture`-Samples

In diesem Abschnitt wird das RTEMS-Sample `capture` analysiert. Im Fokus stehen die Tasks `CT1a`, `CT1b` und `CT1c`, deren Prioritäten sowie die daraus resultierende Ausführungsreihenfolge.

4.1 Task-Erzeugung und Konfiguration

Vor dem Start der Tasks wird ein Mutex mit aktivierter Prioritätsvererbung erzeugt:

```
1 rtems_semaphore_create(
2   name,
3   1,
4   RTEMS_PRIORITY |
5   RTEMS_BINARY_SEMAPHORE |
6   RTEMS_INHERIT_PRIORITY,
7   0,
8   &mutex
9 );
```

Die Option `RTEMS_INHERIT_PRIORITY` aktiviert Priority Inheritance und verhindert Prioritätsinversionen.

Die drei Tasks werden anschließend über `rtems_task_create()` erzeugt.

Task	Priorität	Stack	Modus	Verhalten
CT1a	102	2*1024	RTEMS_PREEMPT, RTEMS_TIMESLICE	Belegt Mutex, wartet, gibt Mutex frei
CT1b	101	2*1024	RTEMS_PREEMPT, RTEMS_TIMESLICE	Busy-Wait-Schleife
CT1c	100	2*1024	RTEMS_PREEMPT, RTEMS_TIMESLICE	Wartet auf Mutex

Tab. 2: Task-Konfiguration im capture-Sample

Da RTEMS kleinere Prioritätswerte als höhere Prioritäten interpretiert, besitzt CT1c die höchste Priorität und CT1a die niedrigste.

4.2 Analyse der Task-Ausführung

Die Tasks werden zeitlich versetzt erzeugt:

```
1  CT1a CREATED
2  CT1a STARTED
3  CT1a SWITCHED_IN
4
5  CT1b CREATED
6  CT1b STARTED
7  CT1b SWITCHED_IN
8
9  CT1c CREATED
10 CT1c STARTED
11 CT1c SWITCHED_IN
```

Obwohl CT1a die niedrigste Priorität besitzt, wird dieser Task zunächst ausgeführt, da zu diesem Zeitpunkt noch keine höher priorisierten Tasks existieren.

Anschließend wird CT1b erzeugt und beginnt eine Busy-Wait-Schleife.

Nach dem Start von CT1c tritt die zentrale Situation des Beispiels auf: CT1c besitzt die höchste Priorität, kann jedoch den Mutex nicht belegen, da dieser noch von CT1a gehalten wird.

Dadurch entsteht eine Prioritätsinversion: Der höher priorisierte Task CT1c wartet auf einen Mutex, der von einem niedriger priorisierten Task gehalten wird, während CT1b CPU-Zeit verbraucht.

4.3 Priority Inheritance

Zur Vermeidung der Prioritätsinversion wurde der Mutex mit RTEMS_INHERIT_PRIORITY erzeugt.

Sobald CT1c blockiert, erbt CT1a temporär die höhere Priorität:

```
1  0a010003 CT1a 102 100 SWITCHED_IN
```

Die erste Zahl beschreibt die ursprüngliche Priorität (102), die zweite die aktuell wirksame Priorität (100).

Dadurch erhält `CT1a` genügend CPU-Zeit, um den Mutex freizugeben. Anschließend wird `CT1c` fortgesetzt und beendet seine Ausführung.

4.4 Zusammenfassung

Das `capture`-Sample demonstriert eine bewusst erzeugte Prioritätsinversion. Durch die Aktivierung von `RTEMS_INHERIT_PRIORITY` wird die Priorität des Mutex-Besitzers temporär angehoben. Der Capture-Trace bestätigt dieses Verhalten und zeigt die korrekte Umsetzung der RTEMS-Prioritätsregeln.

5 Capture-Log-Auswertung

In diesem Abschnitt wird der mit der RTEMS Capture Engine erzeugte Trace des `capture`-Samples analysiert. Ziel ist die Untersuchung der Ereignisse der Tasks `CT1a`, `CT1b` und `CT1c` sowie die Rekonstruktion der Prioritätsvererbung.

5.1 Aufzeichnung des Capture-Logs

Für die Analyse wurde die Capture Engine aktiviert und der Test `test1` zweimal ausgeführt:

```
1-rtems $ copen 50000
1-rtems $ cwglob on
1-rtems $ cwceil 100
1-rtems $ cwfloor 102
1-rtems $ ctset RMON
1-rtems $ cenable

1-rtems $ test1
1-rtems $ test1

1-rtems $ cdisable
1-rtems $ ctrace
```

Die Ausgabe wurde in `Anhang_B_Capture_Log.txt` gespeichert und anschließend gefiltert:

```
grep -E "CT1a|CT1b|CT1c" Anhang_B_Capture_Log.txt \
> capture_trace_only.txt

wc -l capture_trace_only.txt
73 capture_trace_only.txt
```

Damit wurden insgesamt 73 relevante Capture-Records erzeugt. Die Anforderung von mindestens 50 Records für Anhang B ist damit erfüllt.

5.2 Aufbau eines Capture-Records

Ein Capture-Record enthält Informationen über Task-ID, Task-Name, Priorität, Zeitstempel und Ereignistyp.

Wichtige Ereignisse sind:

- `CREATED`: Task erzeugt
- `STARTED`: Task gestartet
- `SWITCHED_IN`: Task erhält CPU
- `SWITCHED_OUT`: Task gibt CPU ab
- `TERMINATED`: Task beendet

Da RTEMS kleinere Zahlen als höhere Prioritäten interpretiert, besitzt `CT1c` mit Priorität 100 die höchste Priorität.

5.3 Analyse von 15 aufeinanderfolgenden Records

Für die Detailanalyse wurden 15 aufeinanderfolgende Records ausgewählt:

```
7 CT1b TASK_RECORD
8 CT1b CREATED
9 CT1b STARTED
10 CT1b SWITCHED_IN
11 CT1b BEGIN
12 CT1b SWITCHED_OUT

13 CT1c TASK_RECORD
14 CT1c CREATED
15 CT1c STARTED
16 CT1c SWITCHED_IN
17 CT1c BEGIN
18 CT1c SWITCHED_OUT

19 CT1b SWITCHED_IN
20 CT1b SWITCHED_OUT
21 CT1a 102 100 SWITCHED_IN
```

Die ersten Einträge zeigen die Erstellung und Ausführung von `CT1b`. Anschließend wird `CT1c` erzeugt und wegen seiner höheren Priorität sofort ausgeführt.

Der Eintrag

```
CT1c SWITCHED_OUT
```

tritt unmittelbar nach dem Versuch auf, den Mutex zu belegen. In Kombination mit dem beobachteten Ablauf deutet dies auf eine Blockierung von `CT1c` am bereits belegten Mutex hin.

Daraufhin läuft zunächst `CT1b` weiter. Anschließend erscheint:

```
CT1a 102 100 SWITCHED_IN
```

Dies zeigt Priority Inheritance: `CT1a` besitzt ursprünglich Priorität 102, wird jedoch temporär mit Priorität 100 ausgeführt.

Die Zeitstempel der Capture-Records zeigen zusätzlich den zeitlichen Abstand zwischen Ereignissen.

Zwischen mehreren aufeinanderfolgenden Records treten nur sehr kleine Zeitdifferenzen auf, was auf unmittelbare Scheduler-Kontextwechsel hinweist.

Größere Zeitabstände entstehen hauptsächlich durch `capture_wait()` sowie durch Blockierungsphasen. Dadurch können die Laufzeiten einzelner Zustände rekonstruiert werden.

5.4 Interpretation der Zustandswechsel

Die ausgewählten Records zeigen die zentrale Situation des Tests: `CT1c` besitzt die höchste Priorität, blockiert jedoch auf einem von `CT1a` gehaltenen Mutex.

Ohne Prioritätsvererbung würde `CT1b` weiterhin CPU-Zeit verbrauchen und `CT1a` könnte den Mutex nicht freigeben. Durch `RTEMS_INHERIT_PRIORITY` wird die Priorität von `CT1a` temporär erhöht.

Dadurch kann `CT1a` den Mutex freigeben und `CT1c` seine Ausführung fortsetzen.

5.5 Gantt-Diagramm der Task-Ausführung

Die zeitliche Task-Ausführung ist in Abbildung 3 dargestellt.

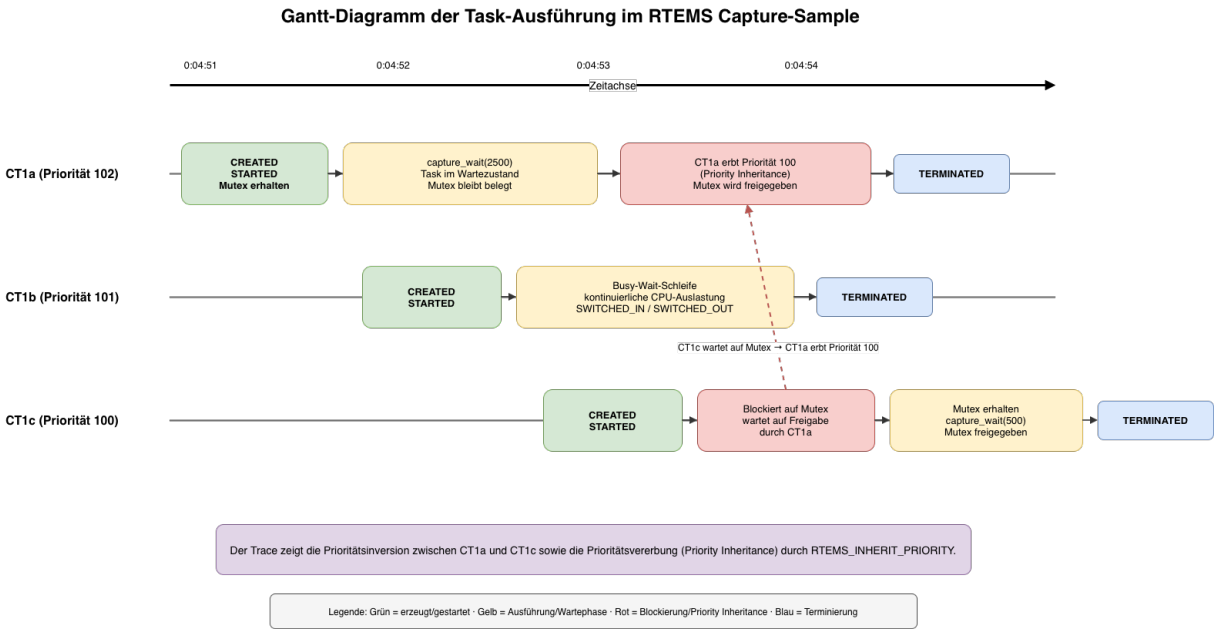


Abb. 3: Gantt-Diagramm der Task-Ausführung im RTEMS `capture`-Sample

Das Diagramm zeigt, dass `CT1c` zunächst blockiert und `CT1a` nach der Prioritätsvererbung erneut CPU-Zeit erhält.

5.6 Zusammenfassung

Die Capture-Analyse bestätigt die zuvor beobachtete Prioritätsinversion. Die ausgewählten Records zeigen die Task-Erzeugung, Kontextwechsel und Priority-Inheritance-Mechanik. Das Verhalten stimmt vollständig mit den RTEMS-Prioritätsregeln und dem erwarteten Ablauf des `capture`-Samples überein.

Das vollständige Capture-Protokoll befindet sich in Anhang B.

6 Vergleich und Fazit

Im abschließenden Abschnitt werden die untersuchten POSIX-Threads aus `psx_example_3` mit der RTEMS Classic API des `capture`-Samples verglichen. Zusätzlich wird das theoretische Verhalten bei identischer Priorität aller CT-Tasks betrachtet und die wichtigsten Erkenntnisse zusammengefasst.

6.1 Vergleich zwischen POSIX-Threads und RTEMS Classic API

Obwohl beide Ansätze parallele Ausführung und Synchronisation unterstützen, unterscheiden sie sich hinsichtlich API-Struktur und Echtzeitverhalten.

Kriterium	**POSIX Threads**	**RTEMS Classic API**
Thread-/Task-Erzeugung	`pthread_create()` erzeugt und startet direkt	`rtems_task_create()` und `rtems_task_start()` getrennt
Synchronisation	Condition Variables, Mutex, Join	Semaphore, Mutex mit Priority Inheritance
Prioritätsverwaltung	Scheduling über `SCHED_FIFO` und Attribute	Direkte numerische Prioritäten im Task-System
Terminierung	`pthread_join()` wartet auf Thread-Ende	`rtems_task_exit()` beendet Task direkt

Tab. 3: Vergleich zwischen POSIX-Threads und RTEMS Classic API

Die RTEMS Classic API bietet einen direkteren Zugriff auf Scheduling- und Echtzeitmechanismen, während POSIX stärker standardisiert und plattformunabhängig aufgebaut ist.

6.2 Theoretischer Ablauf bei gleicher Priorität aller CT-Tasks

Im untersuchten `capture`-Sample besitzen die Tasks unterschiedliche Prioritäten:

- `CT1c`: Priorität 100
- `CT1b`: Priorität 101

- `CT1a`: Priorität 102

Da für alle Tasks zusätzlich

```
1 RTEMS_TIMESLICE
```

aktiviert wurde, würde bei identischen Prioritäten ein Zeitscheibenverfahren verwendet werden.

Dadurch würde sich das Verhalten ändern:

- CPU-Zeit würde zwischen den Tasks aufgeteilt
- `CT1b` könnte die CPU nicht dauerhaft belegen
- `CT1a` würde regelmäßig erneut CPU-Zeit erhalten
- Prioritätsinversion wäre deutlich abgeschwächt

Ein möglicher Ablauf wäre:

1. `CT1a` belegt den Mutex
2. `CT1b` erhält CPU-Zeit
3. `CT1c` blockiert auf dem Mutex
4. durch weitere Zeitscheiben läuft `CT1a` erneut
5. `CT1a` gibt den Mutex frei
6. `CT1c` setzt die Ausführung fort

Da alle Tasks dieselbe Priorität besitzen würden, hätte Priority Inheritance geringere Auswirkungen.

6.3 Fazit und eigene Erkenntnisse

Die Laborübung zeigte deutlich die Unterschiede zwischen allgemeiner POSIX-Thread-Programmierung und der stärker echtzeitorientierten RTEMS Classic API.

Besonders interessant war die Analyse der Prioritätsinversion im `capture`-Sample. Durch die Capture-Auswertung konnte nachvollzogen werden, wie ein höher priorisierter Task blockiert und durch Priority Inheritance dynamisch unterstützt wird.

Die Kombination aus GDB-Debugging, Capture-Logs und Diagrammen ermöglichte nicht nur die Beobachtung einzelner Zustandswechsel, sondern auch ein besseres Verständnis des internen Scheduler-Verhaltens.

Insgesamt vermittelte die Laborübung praktische Einblicke in Echtzeit-Scheduling, Task-Synchronisation und Priority Inheritance innerhalb von RTEMS.